

LECTURE 07 — BUILDING A REAL AI CODE REVIEWER AGENT (HANDS-ON)

"Agent sirf answer dene wala bot nahi hota,\nAgent wo hota hai jo files padhe, soche, decision le aur action kare."

Lecture 07 me humne **actual production-grade AI Code Reviewer Agent** banaya — jo **directory scan karta hai, files read karta hai, bugs fix karta hai aur code overwrite karta hai.**

Is Lecture ka Goal

Is lecture ke baad tum samajh jaoge:

- AI Agent real files ke saath kaise kaam karta hai
- Tools (functions) ko AI ke brain ke saath kaise connect kiya jata hai
- AI se **sirf review nahi, auto-fix kaise karwaya jata hai**

👉 Ye lecture tumhe **Cursor / Devin / GitHub Copilot level thinking** sikhata hai.

High Level Architecture

User → AI Agent → Tools → File System



Decision Making



Code Fix + Write Back

AI:

- Decide karta hai **kaunsi file padhni hai**
- Decide karta hai **kya bug hai**
- Decide karta hai **fix likhna hai ya nahi**

Tools:

- list_files
 - read_file
 - write_file
-

Step 1: Google Gemini AI Setup

```
const ai = new GoogleGenAI({ apiKey: process.env.API_KEY });
```

- ✓ API key `.env` me rakhi → **Security best practice**
 - ✗ Hard-coded key nahi (warna SECURITY BUG hota)
-

Step 2: Tool Functions (Agent ke Hath)

♦ 1. listFiles Tool

Kaam:

- Pure project directory ko recursively scan karta hai
- Sirf relevant files uthata hai

`.js .jsx .ts .tsx .html .css`

🚫 Skip karta hai:

- node_modules
- dist
- build

👍 Ye real-world optimisation hai (warna agent slow ho jata)

♦ 2. readFile Tool

Kaam:

- File content read karna
- AI ko analysis ke liye dena

```
fs.readFileSync(file_path, 'utf-8')
```

👉 Agent bina file padhe **review nahi kar sakta**

♦ 3. writeFile Tool (MOST POWERFUL)

Kaam:

- Fixed code ko wapas disk me likhna

```
fs.writeFileSync(file_path, content)
```

⚠️ Yahin se Agent **dangerous bhi ban sakta hai**
Isliye systemInstruction strong hona zaroori hai.

🧠 Step 3: Tool Registry

```
const tools = {  
  list_files,  
  read_file,  
  write_file  
};
```

👉 AI sirf **registered tools** hi use kar sakta hai

No tool = No action ❌

📜 Step 4: Tool Declarations (AI ko batana)

Yahan hum AI ko formally batate hain:

- Tool ka naam
- Uska kaam
- Parameters ka structure

Example:

```
name: "read_file"  
parameters: { file_path: string }
```

👉 Ye bilkul **API contract** jaisa hota hai

Step 5: System Instruction (AGENT BRAIN)

Ye lecture ka **sabse important hissa** hai.

AI ko humne bola:

Tumhara kaam:

- ① Files list karo
- ② Files read karo
- ③ Analyze karo:

HTML

- Missing alt
- Accessibility issues

CSS

- Invalid syntax
- Duplicate styles

JavaScript

- Bugs
- Security issues
- Console.logs

- ④ Issues FIX karo
- ⑤ write_file se overwrite karo
- ⑥ End me SUMMARY report do

👉 Agent **sirf bolta nahi, karta hai**

Step 6: Agent Loop (Decision Cycle)

```
while (true) {  
  generateContent()  
}
```

AI jab tak:

- Tools call karta rahe → loop continue
- Jab text response de → loop break

👉 Ye hi **Agent lifecycle** hota hai

Real Output Example

 CODE REVIEW COMPLETE

Total Files Analyzed: 1

Files Fixed: 1

🟡 BUG FIXES:

- app.js: Added number validation

🟡 CODE QUALITY:

- Removed console.log

✓ Agent ne file padhi

✓ Bug samjha

✓ Fix likha

✓ Report banayi

Agent vs Normal AI

Normal AI	AI Agent
-----------	----------

Sirf answer	Action leta hai
-------------	-----------------

Stateless	Stateful
-----------	----------

No tools	Tool usage
----------	------------

👉 Agent = AI + Tools + Loop + Memory

Industry Mapping

Is lecture ke baad tum samajh sakte ho:

- Cursor ka reviewer kaise kaam karta hai
 - Devin files kaise modify karta hai
 - GitHub Copilot PR review kaise karta hai
-

Final Thought

"Jab AI file likhne lage —\nsamajh lo tum Agent bana rahe ho,\nchatbot nahi."

Real-World Use Scenarios (Use Cases)

1 Solo Developer / Student

Problem:

- Khud ka code review karne ke liye koi senior nahi
- Small bugs ignore ho jate hain

Agent ka use:

- Project folder ko scan karega
- Common JS / HTML / CSS mistakes fix karega
- Clean report dega

👉 Result: **Senior-level feedback without senior dev**

2 Startup / Small Team

Problem:

- PR reviews slow
- Same type ke bugs baar-baar

Agent ka use:

- Commit se pehle auto-review
- Console.logs, bad practices auto-remove
- Security issues flag

👉 Result: **Fast reviews + consistent quality**

3 Teaching / Course Platform

Problem:

- 100+ students ka code manually check nahi ho sakta

Agent ka use:

- Student project auto-review
- Feedback + fixed version

👉 Result: **Scalable evaluation system**

4 CI/CD Pipeline (Advanced)

Flow:

Git Push → Agent Run → Code Fix → Report → Merge

👉 Future-ready DevOps integration

Detailed Agent Thinking Flow

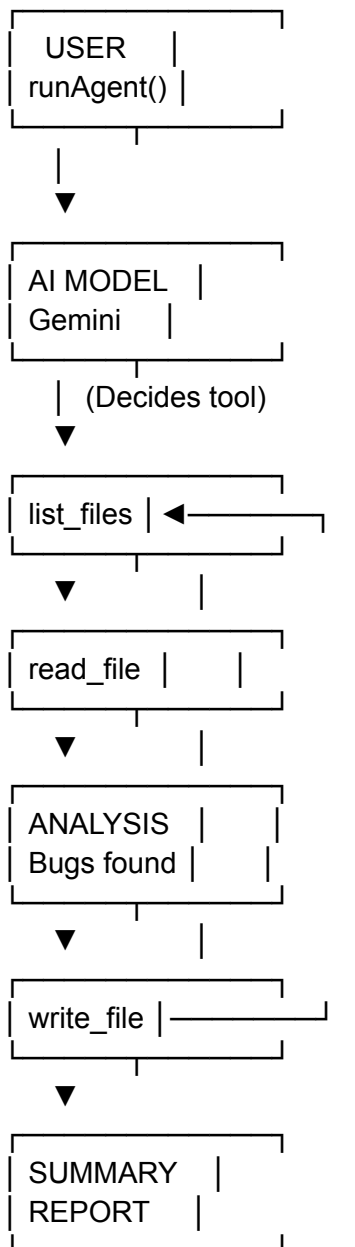
AI internally ye questions poochta hai:

1. Kaun si files important hain?
2. Is file me kya bug / smell hai?
3. Kya fix safe hai?

4. Kya mujhe write_file call karna chahiye?

👉 Ye **decision-making** hi Agent ko powerful banata hai

COMPLETE FLOW DIAGRAM (TEXT)



IMPORTANT SAFETY GUARDRAILS

Agent dangerous bhi ho sakta hai agar:

- write_file blindly use kare
- Secrets overwrite kare

Isliye:

- Strong systemInstruction
 - Limited file extensions
 - Clear FIX-only rule
-

Example Decision Trace

AI: I see console.log → quality issue

AI: Safe to remove

AI: Calling write_file

AI: Hardcoded API key detected

AI: Security risk

AI: Replace with env variable

Socho tum **interviewer / student** ho aur pooch rahe ho:

“Ye agent folder ke andar se ek-ek file kaise nikal raha hai? Step by step kya hota hai?”

CORE QUESTION

Agent folder ke andar se files kaise pick karta hai?

Aur kaise ek-ek file ko dry run karke review karta hai?

Iska answer 3 layers me aata hai:

- 1 Folder scanning algorithm
 - 2 File filtering logic
 - 3 Agent dry-run execution flow
-

◆ PART 1: FOLDER SCANNING ALGORITHM (listFiles)

Ye normal AI nahi karta

Ye real recursive filesystem algorithm hai.

Code ka main idea:

```
scan(directory)
```

Ye function:

- Folder ke andar jata hai
 - Check karta hai: file hai ya folder
 - Agar folder → andar ghus jata hai
 - Agar file → filter lagata hai
-

🧩 ACTUAL ALGORITHM (Human Language)

Step 1

User command:

```
node agent.js ../tester
```

```
➡ directoryPath = ../tester
```

Step 2

Agent AI ko bolta hai:

“Is directory ke code review karo”

AI decide karta hai:

“Mujhe pehle files chahiye”

→ AI calls tool: **list_files**

Step 3

`listFiles({ directory })` start hota hai

`scan("../tester")`

DRY RUN: Folder Traversal

Assume folder structure 📁

```
tester/  
├─ app.js  
├─ index.html  
├─ styles.css  
├─ utils/  
│   ├─ helper.js  
│   └─ test.txt  
├─ node_modules/  
│   └─ react/
```

 **scan("tester")**

❶ **fs.readdirSync("tester")**

Returns:

```
["app.js", "index.html", "styles.css", "utils", "node_modules"]
```

Loop start hota hai

item = "app.js"

- fullPath = tester/app.js
-  node_modules nahi
- `stat.isFile()` → 
- extension = `.js`
- allowed list me hai

 `files.push("tester/app.js")`

item = "index.html"

- file
- `.html` allowed

 push

item = "styles.css"

- `.css` allowed

 push

👉 **item = "utils"**

- directory hai

➡ `scan("tester/utils")`
(recursion starts)

 **scan("tester/utils")**

`fs.readdirSync`
`["helper.js", "test.txt"]`

👉 **helper.js**

- file
- `.js` allowed

✅ `push("tester/utils/helper.js")`

👉 **test.txt**

- `.txt` ❌ allowed list me nahi

🚫 **ignore**

recursion ends

control wapas parent folder me

👉 `item = "node_modules"`

- `fullPath` includes `"node_modules"`

🚫 SKIPPED COMPLETELY

⚠️ Ye bahut important optimisation hai

✅ FINAL FILE LIST

```
[  
  "tester/app.js",  
  "tester/index.html",  
  "tester/styles.css",  
  "tester/utils/helper.js"  
]
```

➡️ Ye array AI ko return hota hai

♦ PART 2: FILE-BY-FILE DRY RUN (Agent Loop)

Ab agent **ek-ek file** ke saath kaam karta hai.

🔄 Agent Loop Logic

```
while (true) {  
  AI thinks  
  if tool call → execute  
  else → finish  
}
```

DRY RUN: app.js

Step 1

AI dekhta hai:

```
files = ["app.js", "index.html", ...]
```

AI decide karta hai:

“Pehle app.js padhta hoon”

➡ AI calls **read_file(app.js)**

Step 2

Tool executes:

```
fs.readFileSync("tester/app.js")
```

➡ content AI ko milta hai

Step 3

AI analysis karta hai:

- console.log?
 - type bug?
 - security issue?
-

Step 4

Agar fix needed:

➡ AI calls **write_file(app.js, fixedCode)**

📌 Is moment pe:

- Real file overwrite hoti hai
 - Agent **ACTION** leta hai
-

Same process repeat hota hai

index.html

- Missing alt?
- Accessibility issue?
- Fix → write_file

styles.css

- Duplicate styles?
- Invalid property?

helper.js

- unused function?
 - bad naming?
-

◆ **PART 3: Agent kab rukta hai?**

Jab AI bole:

“Ab mujhe tool call nahi chahiye
Main summary likhunga”

Tab:

```
result.functionCalls.length === 0
```

- loop BREAK
 - final text output print
-

SIMPLE ONE-LINE LOGIC (Interview Gold)

“Agent pehle recursive scan se relevant files nikalta hai, phir while-loop ke through ek-ek file ko read → analyze → optionally write karta hai, jab tak koi tool call baaki na rahe.”

IMPORTANT OBSERVATION

- ✗ Ye parallel processing nahi hai
- ✓ Ye **sequential, deterministic agent flow** hai

Isliye:

- predictable hai
 - safe hai
 - debuggable hai
-

Interview-Ready Explanation

"This agent uses a tool-based architecture where the LLM decides when to read, analyze, and modify real project files using a controlled loop and explicit function calls."

Why Lecture 07 is IMPORTANT

- Ye lecture **Agent Engineering** ka base hai
 - Iske bina Devin / Cursor samajhna mushkil
 - Ye sirf code nahi, **thinking pattern** sikhata hai
-